

INTERFACES GRÁFICAS DE USUARIO

EJERCICIO PRÁCTICO FINAL

noviembre 2025

¿Listo para controlar los fogones?

Por Óscar Domínguez Álvarez

El presente informe tiene como objetivo presentar el desarrollo y los resultados de la práctica final de Interfaces Gráficas de Usuario. Dicha práctica se centra en la creación de una aplicación para la gestión profesional de un restaurante. A lo largo del documento se describen las diferentes fases del proyecto, desde el análisis de necesidades hasta el diseño, implementación y pruebas del sistema.

Este trabajo no solo ha permitido aplicar los conocimientos adquiridos durante el cuatrimestre, sino también adquirir nuevas competencias en el ámbito del desarrollo de software, la gestión de proyectos y la optimización de procesos en un campo concreto.

Desarrollador

Todo el material del presente documento, así como los archivos adjuntos a él han sido elaborados por Óscar Domínguez Álvarez.

INDICE

INDICE.....	2
1. Introducción	3
2. Planteamiento del problema	4
3. Roadmap	5
Fase 0 - Preparación (día 0: 8 nov)	5
Fase 1 - Modelo de datos y lógica de sesión (día 1: 9 nov)	5
Fase 2 – Ventana principal: sala y selección (días 2-4: 10-12 nov).....	6
Fase 3 – Ventana secundaria no modal (días 5-6: 13-14 nov)	7
Fase 4 – Gestión de estados y comanda (días 7-9: 15-17 nov)	8
Fase 5 – Estadísticas y gráficos en Canvas (días 10-11: 18-19 nov).....	9
Fase 6 – Pruebas finales (día 12: 20-29 nov).....	11
4. Manual del Usuario.....	12
6. Manual del programador.....	14
6.1. Comanda.cs	14
6.2. Mesa.cs	14
6.3. Plato.cs.....	14
6.4. Sesion.cs	14
6.5. MainWindow.xaml.cs.....	15
6.6. GestionComandaWindow.xaml.cs	18
6.7. SecondaryWindow.xaml.cs.....	20
6. Bibliografía.....	21
7. Declaración responsable y uso de IA.....	21

1. Introducción

Este proyecto se desarrolla en el marco de la asignatura Interfaces Gráficas de Usuario, y tiene como objetivo principal el diseño y la implementación de una aplicación de escritorio **WPF (.NET Framework 4.7.2)** para la gestión de un restaurante.

El trabajo se centra especialmente en el desarrollo de la lógica de negocio y en la correcta interacción entre los distintos elementos de la interfaz, priorizando la usabilidad, la claridad y la sincronización entre vistas. El proyecto se ha desarrollado íntegramente en **C#** y **XAML**, siguiendo las restricciones y directrices marcadas en el enunciado de la práctica.

Para organizar el desarrollo de forma estructurada y progresiva, se ha diseñado un **itinerario de trabajo dividido en fases incrementales**. Cada fase representa un avance funcional en la aplicación, abordando desde la creación de las clases que hacen de base a este proyecto hasta la representación gráfica de todas sus vistas.

El desarrollo de la práctica se recoge en el siguiente **planteamiento inicial** y el posterior **roadmap***, que describe las distintas fases del trabajo, sus objetivos y los entregables de cada una. A partir de este esquema, el documento continúa detallando cada fase, explicando las decisiones de diseño adoptadas, el código implementado y las pruebas realizadas en cada etapa del desarrollo.



**Este roadmap ha sido diseñado en un inicio por la IA ChatGPT-5 y modificado posteriormente por el autor de la práctica en función de los cambios y el transcurso de la práctica. Su origen surge a partir de prompts que ofrecían a la IA información sobre el entorno de trabajo, así como los requisitos del enunciado de la práctica.*

2. Planteamiento del problema

Tras leer el enunciado con detenimiento se llega a la conclusión de que lo primero que se va a necesitar es una clase **Sesion** que se comporte como núcleo del sistema.

Esta sesión contendrá las mesas activas, el menú del restaurante que, en esencia, serán un conjunto de platos y el historico de las comandas. En nuestro caso, se ha decidido plantear un restaurante con **12 posiciones fijas** (en el contexto del código, celdas) por lo que se necesitará una matriz (*Disposicion*) que facilitará el trabajo de dibujar y trabajar con las mesas.

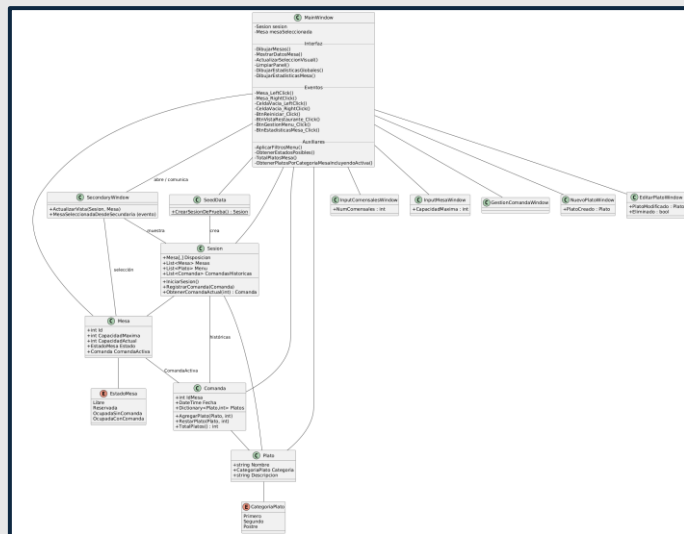
Esta sesión será en un principio “hardcodeada” y se creará a mano en una clase que llamada **SeedData**. Este kit de juego inicial servirá para hacer pruebas rápidas y tener un punto de partida en la sala del restaurante.

Como motor visual se desarrollará un **MainWindow**, desde el cual se lanzarán otras ventanas auxiliares y la “Vista del Restaurante”, uno de los requisitos primordiales de la práctica. Esta ventana cargará el SeedData nada más empezar y tras ello deberá ofrecer los medios oportunos para satisfacer las necesidades del programa.

El planteamiento de cara a esta clase *MainWindow* es hacer un *canvas* con un panel lateral que contenga toda la información de una mesa (cuando esta se encuentre seleccionada) y que contenga todos botones que hagan de pasarela a otras vistas.

Para el control y gestión de las mesas, se crearán controladores de eventos sobre ellas, (con los clicks del ratón) que muestren un *ContextMenu*.

Con esta idea general, se ha desarrollado el siguiente **diagrama*** que servirá como punto de partida para la realización de práctica.



Teniendo este punto de partida y las bases sentadas, es posible avanzar al planteamiento de un itinerario (roadmap) que muestre el desarrollo de lo planteado

*Este diagrama se ha entregado junto con el resto de los documentos en formato .png por si es necesario observarlo con más detalle.

**El desarrollo de este diagrama se realizó al comienzo de la práctica como planteamiento inicial, por lo que es posible que los archivos .cs finales tengan variaciones o ampliaciones de cara a atributos, clases o métodos.

3. Roadmap

Roadmap por fases (incremental) – objetivo entrega 02-dic-2025

Situación inicial: 8-nov-2025. **Tiempo real hasta objetivo:** 24 días.

Fase 0 - Preparación (día 0: 8 nov)

Duración: 1 día

Objetivo: preparar proyecto, estructura y convenios.

Tareas:

- Crear solución WPF (.NET Framework) en Visual Studio: TrabajoIGU.sln.
- Crear proyectos / carpetas: Models/, Data/. para ordenar el contenido
- Crear clases iniciales de dominio (esqueleto): Mesa, Comanda, Plato, Sesion.
- Crear un repositorio en GitHub para poder gestionar los avances y recoger todos los archivos con cada avance.

Entregable: solución vacía con estructura de carpetas y esqueleto de clases.

Fase 1 - Modelo de datos y lógica de sesión (día 1: 9 nov)

Duración: 1 día

Objetivo: **(Requisito 8)** implementar las clases base, así como el SeedData.

Tareas:

- Implementar Mesa con Id, CapacidadMaxima, CapacidadActual y Estado (enum: Libre, Reservada, OcupadaSinComanda, OcupadaConComanda).
- Implementar Plato con Nombre, Categoría y Descripción.
- Implementar Comanda (lista de Platos + cantidades + referencia a Mesa). Asegurar almacenamiento por sesión (no perder comandas cuando mesa vuelve a Libre (guardarlas en el histórico).
- Implementar Sesion con lista de mesas, Menu (platos disponibles), historial de comandas por mesa; métodos IniciarSesion() y métodos auxiliares para obtener comandas y registrarlas en el histórico.
- Crear datos de prueba cargados al arranque SeedData (Requisito 8: ≥ 5 mesas, ≥ 3 platos por categoría, mesas con capacidadActual ≥ 2 , sesión con ≥ 3 mesas ocupadas y comandas, al menos una mesa con 2 grupos consecutivos).

Entregable: Models funcional + datos de prueba en el arranque.

Fase 2 – Ventana principal: sala y selección (días 2-4: 10-12 nov)

Duración: 3 días

Objetivo: **(Requisito 1, 4 y 5)** diseñar y desarrollar la ventana principal de la aplicación en WPF, representando la sala del restaurante mediante un Canvas. Implementar la gestión visual e interactiva de las mesas, el área de información del restaurante y las acciones principales de usuario, así como una función que permita el reinicio de la sesión.

Tareas:

- Dibujar la sala con Canvas:
 - Crear una disposición de 4 filas × 3 columnas (12 posiciones).
 - Representar las mesas existentes con imágenes según su estado.
 - Dibujar marcos transparentes con número de celda en los huecos vacíos.
 - Asignar fondo con textura del suelo del restaurante.
- Gestión de eventos:
 - Click izquierdo para seleccionar/deseleccionar mesas o limpiar el panel si se hace en celdas vacías.
 - Click derecho sobre mesas: menú contextual con opciones para cambiar estado, editar número de comensales y eliminar mesa.
 - Click derecho sobre celdas vacías: menú contextual para añadir nuevas mesas.
 - Controlar flujos lógicos de estados (Libre ↔ Reservada ↔ Ocupada ↔ Ocupada con comanda).
- Información de restaurante y mesas:
 - Panel lateral con:
 - Título "Información del Restaurante".
 - Mesas activas y aforo dinámico (personas actuales / capacidad total).
 - Datos de la mesa seleccionada (ID, capacidad, estado, comensales actuales).
 - Contador de platos de la comanda actual.
 - Botón de "Reiniciar" para limpiar la sesión.
- Mejoras funcionales
 - Selección visual de la mesa (borde rojo).
 - Al crear mesa nueva, queda automáticamente seleccionada.
 - Validaciones:
 - No permitir comanda sin comensales.
 - Si mesa pasa a "Libre", se reinician comensales a 0.

- Al poner comensales en 0 estando con comanda, advertir y liberar mesa.

Entregable: Ventana principal completamente funcional que muestra la sala del restaurante, permite gestionar mesas (añadir, eliminar, cambiar estado, editar comensales) y mantiene actualizada la información del restaurante. Todo integrado visualmente con imágenes y validaciones lógicas según el flujo de trabajo del restaurante.

Fase 3 – Ventana secundaria no modal (días 5-6: 13-14 nov)

Duración: 2 días

Objetivo: **(Requisito 2 y 3)** diseñar e implementar la ventana secundaria no modal, estructurada en dos tablas, y garantizar la sincronización bidireccional de selección entre la ventana principal y la secundaria.

Tareas:

- Implementar SecondaryWindow.xaml (no modal) con:
 - Tabla 1: listado de mesas (Id, capacidad máxima, estado, número real de comensales si está ocupada).
 - Tabla 2: listado de platos de la comanda de la mesa seleccionada (categoría, nombre, descripción y número de unidades).
 - Si la mesa no tiene comanda activa o no está en estado OcupadaConComanda, se muestra un mensaje informativo: "No se ha seleccionado una mesa con comanda".
- Sincronización bidireccional entre ventanas:
 - Al seleccionar una fila en la tabla de mesas de SecondaryWindow, se emite un evento (MesaSeleccionadaDesdeSecundaria) que actualiza la mesa seleccionada en MainWindow y su respectivo panel de información.
 - Cuando se selecciona una mesa en el canvas de la ventana principal, SecondaryWindow actualiza la fila correspondiente, refresca las tablas si es necesario y mantiene la selección visual aun perdiendo el foco (cuando se hace click en la MainWindow).

Entregable: Ventana secundaria completamente funcional, no modal, integrada visual y funcionalmente con la principal con sincronización bidireccional entre ambas vistas, selección estable, persistente y correctamente resaltada en ambas ventanas y arquitectura preparada para integrarse sin cambios en fases posteriores (comandas, estadísticas, etc.).

Fase 4 – Gestión de estados y comanda (días 7-9: 15-17 nov)

Duración: 3 días

Objetivo: **(Requisito 5 y 6)** implementar un sistema robusto de gestión de estados por mesa y un flujo completo para creación, edición y finalización de comandas, garantizando coherencia lógica y persistencia de datos en la sesión.

Tareas:

- Mejorar el sistema de cambio de estado mediante menú con validaciones:
Validaciones incluidas:
 - No se puede abrir una comanda con CapacidadActual = 0.
 - Al pasar a EstadoMesa.Libre, si la mesa tenía comanda activa, se pregunta al usuario y se finaliza adecuadamente.
 - Cambios en comensales pueden modificar automáticamente el estado (por ejemplo, si queda en 0).
- Comanda activa por mesa + historial separado
 - Se añadió Mesa.ComandaActiva, que representa la comanda en curso.
 - El histórico (Sesion.ComandasHistoricas) contiene únicamente comandas finalizadas.
 - Se evita la reutilización errónea de comandas antiguas en mesas reocupadas.
- Ventana de gestión de comandas (crear / editar)
 - Ventana modal GestionComandaWindow con:
 - Selector de platos organizados por categorías.
 - Doble click para añadir plato.
 - Tarjetas compactas para platos añadidos con botones + y -.
 - Filtros por texto y categoría.
 - Estado visual cuidado (selección pegada, tarjetas unificadas, etc.).
 - Comportamientos implementados:
 - Crear nueva comanda desde cero.
 - Editar comanda activa existente.
 - Guardar cambios.
 - Cancelar con aviso si hay modificaciones sin guardar.
 - Detección automática de "comanda vacía".
- Lógica estricta para creación y eliminación de comandas. Casos controlados:
 - Mesa sin comanda activa → Crear nueva
 - Si no se añaden platos → NO se crea comanda.
 - Se avisa al usuario antes de descartar.
 - Mesa con comanda activa → Editar

- Si se deja vacía → se elimina la comanda y se cambia a OcupadaSinComanda.
 - Si tiene platos → se actualiza correctamente.
- Finalización explícita de comanda
 - Nuevo botón contextual Finalizar comanda.
 - Mueve la comanda al histórico.
 - Limpia ComandaActiva.
 - Pone la mesa en Libre y pone CapacidadActual = 0.
- Actualización dinámica de la información de la mesa
 - El panel lateral ahora utiliza SIEMPRE mesa.ComandaActiva para mostrar número de platos.
 - Se evita el error anterior de leer comandas antiguas.
 - La ventana secundaria se sincroniza automáticamente.

Entregable: Sistema completo de cambios de estado, validado y coherente con gestión completa de comandas (crear, editar, finalizar). Separación correcta entre comanda activa y comandas históricas y persistencia adecuada para estadísticas futuras.

Fase 5 – Estadísticas y gráficos en Canvas (días 10-11: 18-19 nov)

Duración: 2 días

Objetivo: **(Requisito 7)** representar correctamente los gráficos estadísticos mediante Canvas y manteniendo la integración con la lógica de comandas y selección de mesa.

Tareas:

- Gráfico global de mesas: Cuando no hay ninguna mesa seleccionada, se muestra en la ventana principal un gráfico de columnas donde:
 - Se genera una leyenda explicativa
 - Cada columna representa una mesa.
 - La altura corresponde al total de platos consumidos calculado a partir de:
 - comandaActiva de la mesa (si existe),
 - comandas históricas de esa mesa.
 - Las mesas “existentes” (con objeto Mesa activo) se dibujan en azul.
 - Las mesas no “existentes”, (pero con comandas en el histórico) se dibujan en gris
 - Se muestran etiquetas de texto con el número total de platos encima de cada barra.

- Gráfico por mesa seleccionada: Si hay una mesa seleccionada:
 - Se muestran tres columnas: Primeros, Segundos, Postres.
 - Cada columna se segmenta en varios rectángulos:
 - Cada segmento representa un plato concreto.
 - Altura proporcional al nº de veces que ese plato aparece en todas las comandas (histórico + activa).
 - Color único por plato.
 - Etiquetas con la cantidad encima de cada segmento.
 - Se genera automáticamente una leyenda de colores con nombre del plato.

- Actualización automática. Los gráficos se actualizan cada vez que:
 - Se selecciona una mesa distinta.
 - Se crea o edita una comanda.
 - Se elimina o finaliza una comanda.
 - Se añaden nuevas mesas o platos.
 - Se redimensiona la ventana.

- Integración con panel lateral
 - Estadísticas globales
 - Oculta la vista de mesas y muestra el gráfico de todas las mesas.
 - Estadísticas de la mesa
 - Se activa sólo si hay mesa seleccionada.
 - Muestra sus estadísticas individuales.

Entregable: Gráfico global con todas las mesas y registro del total de platos histórico + actual; Gráfico segmentado por categorías de una mesa; Botones y navegación entre sala y estadísticas; Actualización automática en tiempo real del canvas de las estadísticas de mesa; Resolución de casos conflictivos (mesas sin comandas o mesaSeleccionada=null).

Fase 6 – Pruebas finales (día 12: 20-29 nov)

Duración: 10 días

Objetivo: verificar requisitos del enunciado, añadir mejoras opcionales que vayan surgiendo, corregir errores y preparar el entregable final.

Tareas:

- Implementación de enlaces (sustitución de comunicación entre ventanas por eventos)
- Añadir .ico como icono del ejecutable
- Añadir persistencia con archivos de texto:
 - Cargar menú de un .txt
 - Generar factura una vez finalizada una comanda
 - Se inicializa una ventana desde la cual el usuario puede decidir donde guardarla.
- Limpiar comentarios (Borrarlos o darles un formato uniforme y más legible de cara a la defensa)
- Añadir botón vaciar mesas
- Añadir guías a las estadísticas

Entregable: build estable para defensa (ejecutable + código mejorado + documentación).

4. Manual del Usuario

Canvas (Sala del Restaurante)

- Click Dcho sobre mesa
 - Si mesa.Estado es libre
 - Cambiar mesa.Estado a Reservada / OcupadaSinComanda
 - Eliminar mesa
 - Si mesa.Estado es reservada
 - Cambiar mesa.Estado a OcupadaSinComanda / Libre
 - Editar nº comensales actuales
 - Eliminar mesa
 - Si mesa.Estado es OcupadaSinComanda
 - Cambiar mesa.Estado a OcupadaConComanda / Libre
 - Editar nº comensales actuales
 - Editar comanda
 - Eliminar mesa
 - Si mesa.Estado es OcupadaConComanda
 - Cambiar mesa.Estado a Libre
 - Editar nº comensales actuales
 - Editar comanda
 - Finalizar comanda
 - Eliminar mesa

*Casos y requisitos especiales (Click Dcho sobre Mesa)

- Será necesario tener comensales en una mesa para abrir una comanda.
- Será necesario tener comanda para estar en estado OcupadaConComanda.
- Será necesario tener comensales en la mesa para estar en estado OcupadaConComanda.
- Si el estado pasa de OcupadaConComanda a Libre o bien cambian los comensales actuales a 0, se dará por finalizada la comanda y se guardará en el registro. De la misma manera, finalizar la comanda establecerá el estado Libre en la mesa y el número de comensales a 0.
- Será necesario tener comensales actuales para editar una comanda en estado OcupadaSinComanda.
- Editar una comanda en estado OcupadaSinComanda modificará el estado de la mesa a OcupadaConComanda.
- Eliminar una mesa sin finalizar la comanda, eliminará la comanda actual notificando previamente al usuario sobre su decisión.
- No se podrá crear una comanda con 0 platos
- Eliminar todos los platos de una comanda supondrá una vuelta al estado OcupadaSinComanda

- Click Izqdo sobre mesa
 - Seleccionar una mesa (Se indica con cambio de color del marco de la mesa a rojo)
- Click Dcho sobre celda vacía
 - Añadir una mesa
- Click Izqdo sobre canvas/celda vacía
 - Quitar la selección de una mesa

Panel Lateral

- Botón Estadísticas Globales
- Botón Gestión del Menú
- Botón Vista del Restaurante
- Botón Iniciar/Reiniciar Sesión
- Botón Estadísticas de la mesa

Estadísticas Globales

- Botón Volver

Gestión del Menú

- Botón Volver
- Botón + Añadir plato permite añadir un nuevo item
- Expander (Buscador por filtros)
 - TextBox (Filtro texto)
 - ComboBox (Filtro categoría)
 - Botón Limpiar filtros
- Doble click sobre item permite editarlo

Vista del Restaurante

- Selección de row (fila) dentro del data grid con conexión bidireccional con la sala del restaurante

Vaciar Mesas

- Permite vaciar las mesas (ComensalesActuales=0, Estado=Libre, etc.)

Reiniciar Sesión

- Permite vaciar el restaurante (Dejarlo sin mesas)

Estadísticas de la Mesa

- Botón Volver

6. Manual del programador

Manual de las principales funciones de cada archivo:

6.1. Comanda.cs

- **Comanda(int idMesa)**
Constructor. Inicializa una comanda nueva para la mesa indicada, con fecha actual y diccionario de platos vacío.
- **int TotalPlatos()**
Suma y devuelve el número total de platos (sumando cantidades) de la comanda.
- **void AñadirPlato(Plato plato)**
Aumenta en 1 la cantidad del plato dado (o lo añade si no estaba) y notifica cambio.
- **void QuitarPlato(Plato plato)**
Resta 1 a la cantidad del plato, y lo elimina del diccionario si llega a 0; notifica cambio.
- **void OnPropertyChanged(string propertyName)**
Lanza el evento PropertyChanged para avisar a la interfaz de cambios en la comanda.

6.2. Mesa.cs

- **Mesa(int id, int capacidadMaxima)**
Constructor. Crea una mesa con id, capacidad máxima, estado libre y sin comanda activa.
- **OnPropertyChanged(string propertyName)**
Lanza el evento PropertyChanged para notificar cambios en propiedades como estado, comensales, selección o comanda activa. *(Las propiedades CapacidadActual, Estado, MesaSeleccionada y ComandaActiva usan este método cada vez que cambian su valor.)*

6.3. Plato.cs

- **Plato(string nombre, CategoriaPlato categoria, string descripcion)**
Constructor. Crea un plato con nombre, categoría y descripción opcional.

6.4. Sesion.cs

- **Sesion()**
Constructor. Crea una sesión nueva con matriz de mesas vacía, lista de mesas vacía, menú vacío y sin comandas históricas.

- **void IniciarSesion()**
Reinicia la sesión: limpia la disposición de mesas, reinicia la lista de mesas y borra el historial de comandas (mantiene el menú).
- **Comanda ObtenerComandaActual(int idMesa)**
Devuelve la comanda activa de la mesa con ese id (a través de la propiedad ComandaActiva de la mesa).
- **void SeleccionarMesa(Mesa mesaSeleccionada)**
Marca una única mesa como seleccionada (propiedad MesaSeleccionada) y desmarca el resto. Si se pasa null, deselecciona todas las mesas.
- **void AñadirMesa(Mesa nuevaMesa, int fila, int columna)**
Coloca una nueva mesa en la matriz Disposicion en la posición indicada, la añade a la lista Mesas y notifica que la disposición ha cambiado.
- **void EliminarMesa(Mesa mesaAEliminar)**
Quita la mesa de la matriz Disposicion y de la lista Mesas y notifica cambio de disposición.
- **OnPropertyChanged(string propertyName)**
Lanza el evento PropertyChanged para avisar a las ventanas que la sesión ha cambiado (por ejemplo, la matriz Disposicion).

6.5. MainWindow.xaml.cs

Inicialización / interfaz

- **MainWindow()**
Constructor. Crea la sesión de prueba (SeedData), inicializa la ventana y dibuja las mesas.
- **void DibujarMesas()**
Calcula el tamaño y la posición de cada celda en el Canvas, dibuja las mesas o celdas vacías, asigna eventos y etiqueta los números de mesa.
- **void canvasSala_SizeChanged(object sender, SizeChangedEventArgs e)**
Cuando cambia el tamaño del Canvas de la sala, vuelve a dibujar las mesas de forma proporcional.
- **void ActualizarSeleccionVisual()**
Recorre todas las mesas dibujadas y pinta el borde en rojo si es la mesa seleccionada, negro en caso contrario.
- **void MostrarDatosMesa()**
Actualiza el panel lateral con el resumen del restaurante (aforo) y los datos básicos y número de platos de la mesa seleccionada.

- **void LimpiarPanel()**
Rellena el panel lateral con el texto "Ninguna mesa seleccionada" en todos los campos.

Estadísticas y gráficos

- **void DibujarEstadisticasGlobales()**
Dibuja en el canvasEstadisticasGlobales el gráfico de barras por mesa (platos totales) y la leyenda de colores azul/gris.
- **void canvasEstadisticasGlobales_SizeChanged(object sender, SizeChangedEventArgs e)**
Vuelve a dibujar el gráfico global cuando cambia el tamaño del canvas.
- **int TotalPlatosMesa(Mesa mesa)**
Calcula el total de platos consumidos en una mesa sumando historial + comanda activa.
- **void DibujarEstadisticasMesa()**
Dibuja el gráfico de la mesa seleccionada: tres columnas (primeros, segundos, postres), segmentadas por plato, con leyenda de colores.
- **void canvasEstadisticasMesa_SizeChanged(object sender, SizeChangedEventArgs e)**
Recalcula el gráfico de la mesa seleccionada al cambiar el tamaño del canvas.
- **Brush GetColorParaPlato(string nombrePlato)**
Asigna un color consistente a cada plato y lo guarda en mapaColoresPlatos para reutilizarlo.
- **Dictionary<string,int> ObtenerPlatosPorCategoriaMesaIncluyendoActiva(Mesa mesa, CategoriaPlato categoria)**
Devuelve, para una mesa y categoría, cuántas veces se ha pedido cada plato (histórico + comanda activa).

Persistencia / facturas

- **static void GenerarFactura(Mesa mesa, Comanda comanda)**
Genera el texto de la factura (mesa, fecha, platos y cantidades) y abre un SaveFileDialog para que el usuario elija dónde guardar el .txt.

Suscripciones y selección

- **void SuscribirseMesasSesion()**
Recorre todas las mesas de la sesión, les quita y les añade el manejador MesaSeleccionadaOnChanged, y guarda las suscritas en mesasSuscritas.

- **void Mesa_LeftClick(object sender, MouseButtonEventArgs e)**
Cuando se hace click izquierdo sobre el borde de una mesa, la marca como seleccionada, actualiza la sesión, el panel lateral y la selección visual.
- **void Mesa_RightClick(object sender, MouseButtonEventArgs e)**
Abre el menú contextual de una mesa (cambiar estado, editar comensales, editar comanda, finalizar comanda, eliminar mesa) aplicando toda la lógica de estados, facturas y guardado de comanda.
- **void Canvas_MouseLeftButtonDown(object sender, MouseButtonEventArgs e)**
Si se hace click en una zona vacía del Canvas, deselectiona la mesa actual y limpia el panel.
- **void ComandaOnChanged(object sender, PropertyChangedEventArgs e)**
Cuando cambia la comanda activa de la mesa seleccionada, actualiza los datos mostrados de la mesa.
- **void MesaSeleccionadaOnChanged(object sender, PropertyChangedEventArgs e)**
Actualiza mesaSeleccionada cuando alguna mesa cambia su propiedad MesaSeleccionada, refresca panel, selección y gráfico de mesa.

Botones de sesión y sala

- **void BtnReiniciar_Click(object sender, RoutedEventArgs e)**
Pide confirmación y, si se acepta, reinicia la sesión completa (borra mesas e historial), limpia selección y redibuja.
- **void BtnVaciarMesas_Click(object sender, RoutedEventArgs e)**
Pide confirmación y deja todas las mesas activas en estado Libre, con 0 comensales y sin comanda activa.
- **void BtnVistaRestaurante_Click(object sender, RoutedEventArgs e)**
Abre la SecondaryWindow (o la trae al frente si ya está abierta) con la sesión y la mesa actual.

Navegación de vistas (gestión menú / estadísticas)

- **void BtnGestionMenu_Click(object sender, RoutedEventArgs e)**
Oculta el grid del restaurante, muestra el de gestión de menú y aplica filtros por defecto.
- **void BtnVolverGestionMenu_Click(object sender, RoutedEventArgs e)**
Vuelve de la vista de gestión de menú a la vista del restaurante.

- **void BtnEstadisticasGlobales_Click(object sender, RoutedEventArgs e)**
Cambia de la vista de restaurante a la vista de estadísticas globales y dibuja el gráfico.
- **void BtnVolverEstadisticasGlobales_Click(object sender, RoutedEventArgs e)**
Vuelve de estadísticas globales a la vista del restaurante.
- **void BtnEstadisticasMesa_Click(object sender, RoutedEventArgs e)**
Si hay mesa seleccionada, abre la vista de estadísticas de esa mesa y dibuja el gráfico correspondiente.
- **void BtnVolverEstadisticasMesa_Click(object sender, RoutedEventArgs e)**
Vuelve de la vista de estadísticas de mesa al restaurante.

Gestión del menú (alta/baja/modificación de platos)

- **void BtnAgregarPlatoMenu_Click(object sender, RoutedEventArgs e)**
Abre la ventana NuevoPlatoWindow; si el usuario guarda, añade el nuevo plato al menú y refresca la lista.
- **void FiltroMenu_Changed(object sender, EventArgs e)**
Vuelve a aplicar filtros de menú cuando cambia el texto o la categoría.
- **void BtnLimpiarFiltrosMenu_Click(object sender, RoutedEventArgs e)**
Limpia los filtros de búsqueda y deja el combo de categoría en "Todas".
- **void LvPlatosMenu_MouseDoubleClick(object sender, MouseButtonEventArgs e)**
Abre la ventana EditarPlatoWindow para el plato seleccionado; según la acción, lo modifica o lo elimina y refresca el menú.
- **void LvPlatosMenu_MouseLeftButtonDown(object sender, MouseButtonEventArgs e)**
Si se hace click en una zona vacía de la lista, deselecta el plato del menú.

6.6. GestionComandaWindow.xaml.cs

- **GestionComandaWindow(Sesion sesion, Mesa mesa)**
Constructor. Recibe la sesión y la mesa, actualiza el título con el número de mesa, carga la comanda actual y el menú.
- **void CargarMenu()**
Carga en lvMenu la lista de platos del menú ordenados por categoría.

- **void CargarComanda()**
Copia el estado actual de la comanda activa de la mesa en comandaOriginal y refresca la lista de platos de la comanda.
- **void RefrescarComanda()**
Vuelve a enlazar lvComanda con los platos de mesa.ComandaActiva (para que la UI muestre cantidades actualizadas).
- **void BtnMas_Click(object sender, RoutedEventArgs e)**
Evento del botón + en cada tarjeta de comanda: añade una unidad de ese plato y marca que hay cambios.
- **void BtnMenos_Click(object sender, RoutedEventArgs e)**
Evento del botón -: resta una unidad del plato y lo elimina si llega a 0; marca que hay cambios.
- **void LvMenu_MouseDoubleClick(object sender, MouseButtonEventArgs e)**
Doble click en un plato del menú: añade una unidad de ese plato a la comanda activa y refresca la vista.
- **void BtnGuardar_Click(object sender, RoutedEventArgs e)**
Marca que el cierre viene desde "Guardar" y cierra la ventana (la lógica de qué hacer después está en MainWindow).
- **void BtnCancelar_Click(object sender, RoutedEventArgs e)**
Cierra la ventana sin marcar guardado; el cierre pasará por OnClosing.
- **override void OnClosing(CancelEventArgs e)**
Pregunta al usuario si quiere descartar cambios cuando se han hecho modificaciones y no se ha pulsado "Guardar"; si cancela, no cierra la ventana y restaura la comanda original.
- **bool ComandasSonIguales()**
Compara la comanda actual (mesa.ComandaActiva.Platos) con comandaOriginal para saber si ha habido cambios.
- **void AplicarFiltros()**
Aplica filtros de texto y categoría sobre el menú y actualiza lvMenu con los platos filtrados.
- **void Filtro_Changed(object sender, RoutedEventArgs e)**
Manejador genérico cuando cambia el texto o la categoría: vuelve a aplicar los filtros.
- **void BtnLimpiarFiltros_Click(object sender, RoutedEventArgs e)**
Limpia el cuadro de búsqueda, reinicia la categoría a "Todas" y vuelve a mostrar todo el menú.

6.7. SecondaryWindow.xaml.cs

- **SecondaryWindow(Sesion sesionActiva, Mesa mesaSeleccionada)**
Constructor. Recibe la sesión y la mesa seleccionada, se suscribe a cambios de la sesión y dibuja la vista inicial.
- **void ActualizarVista()**
Vuelve a suscribirse a las mesas y a la comanda de la mesa actual, rellena el DataGrid de mesas y selecciona la mesa actual, luego actualiza el DataGrid de platos.
- **void SuscribirseMesasSesionSeleccionada()**
Gestiona las suscripciones al evento PropertyChanged de todas las mesas, para reaccionar a cambios de selección o de comanda.
- **void SuscribirseComandaDeMesaActual()**
Gestiona la suscripción a la ComandaActiva de la mesa actual para actualizar la tabla de platos cuando cambie.
- **void ActualizarPlatos()**
Rellena o limpia dgPlatos según si hay mesa/comanda activa; muestra un mensaje cuando no hay comanda.
- **void dgMesas_SelectionChanged(object sender, SelectionChangedEventArgs e)**
Cuando el usuario selecciona una mesa en el DataGrid, actualiza el campo mesa y llama a sesion.SeleccionarMesa(...).
- **void dgMesas_MouseLeftButtonDown(object sender, MouseButtonEventArgs e)**
Si el click es en un área vacía, deselectiona la mesa y avisa a la sesión de que no hay mesa seleccionada.
- **void ComandaOnChanged(object sender, PropertyChangedEventArgs e)**
Cuando cambia la comanda suscrita, vuelve a llamar a ActualizarPlatos().
- **void SesionOnChanged(object sender, PropertyChangedEventArgs e)**
Si cambia la sesión (por ejemplo, la disposición), vuelve a actualizar la vista completa.
- **void MesaSeleccionadaOnChanged(object sender, PropertyChangedEventArgs e)**
Responde a cambios en Mesa.MesaSeleccionada o Mesa.ComandaActiva: actualiza qué mesa mostrar y vuelve a pintar.
- **override void OnClosed(EventArgs e)**
Limpia todas las suscripciones (PropertyChanged) cuando se cierra la ventana para evitar fugas de eventos.

6. Bibliografía

Apuntes de clase

- Principalmente usados:
TEMA 4 (Eventos), TEMA 6 (Selectores), TEMA 7 (Cuadros de mensaje y diálogo), TEMA 8 (Enlaces)

Documentación externa

- Apertura y uso de archivos (Persistencia):
https://www.w3schools.com/cs/cs_files.php
- Uso de Dictionary (kvp):
<https://learn.microsoft.com/es-es/dotnet/api/system.collections.generic.dictionary-2?view=net-10.0>

Uso de IA

- ChatGPT 5.1

7. Declaración responsable y uso de IA

El modelo empleado principalmente en la realización de esta práctica ha sido ChatGPT 5. Se ha empleado diariamente en el transcurso de esta práctica y ha sido la herramienta principal para todo tipo de tareas.

Gestión y usos de esta IA en el proyecto:

- **Organización:** ChatGPT proporcionó un roadmap con el cual se estableció una organización temporal, incremental e iterativa, que no solo ayudó a estructurar la documentación, sino que también sirvió de bases para comenzar la entrega.
- **Autocompletación de código:** Dados unos prompts o comentarios sobre el código del autor, se le pidió a la IA que autocompletara fragmentos de código para llevar a cabo la tarea oportuna. Esto ayudó a aquellas partes en las que había desconocimiento total en el ámbito de desarrollo de C# / XAML y permitió al alumno conocer nuevas herramientas y recursos además de avanzar y lograr la finalidad del prompt. Sean ejemplos: expresiones Lambda, uso de var, Dictionary<UIElement, Mesa>, funciones auxiliares, propiedades adicionales de XAML, etc.
- **Información:** ChatGPT fue empleado como herramienta de ampliación de conocimiento. Esta ayudó explicando y aclarando conceptos clave de la práctica.